T05

PROGRAMACIÓN ORIENTADA A OBJETOS:HERENCIA (II)

MIEMBROS PÚBLICOS VS NO PÚBLICOS

- Python tiene una nomenclatura que debemos utilizar para comunicar que un atributo o método no está diseñado para usarse desde fuera de la clase u objeto que lo contiene.
- La convención de nomenclatura consiste en agregar, uno o dos guiones bajos al nombre del miembro.
- En Python, los atributos y métodos públicos de una clase constituyen lo que conocerá como la interfaz de la clase o interfaz de programación de aplicaciones (API).
- Por otro lado, los miembros protegidos solo deben ser accedidos desde la propia clase o sus subclases (aunque esto es una convención, ya que en la práctica sí pueden ser accedidos).
- Finalmente, un miembro privado no puede ser accedido desde fuera de la clase.
- En una clase de Python, tendremos la siguiente convención:

Miembro	Nomenclatura	Accesible desde	Ejemplos
Público	Patrón de nomenclatura normal	Cualquier clase (o script)	radio, calcular_area()
Protegido	_ inicial en los nombres	La propia clase y sus sub-clases	_radio, _calcular_area ()
Privado	__ inicial en los nombres	La propia clase	__radio, __calcular_area()

MIEMBROS PÚBLICOS VS NO PÚBLICOS

```
class Padre:
    def __init__(self):
        self.__privado = 'privado'
        self._protegido = 'protegido'
        self.publico = 'publico'

    def accede(self):
        print(f'{self.__class__.__name__} - {self.publico}')
        print(f'{self.__class__.__name__} - {self._protegido}')
        print(f'{self.__class__.__name__} - {self.__privado}')

class Hijo(Padre):
    def accede(self):
        print(f'{self.__class__.__name__} - {self.publico}')
        print(f'{self.__class__.__name__} - {self._protegido}')
        # print(f'{self.__class__.__name__} - {self.__privado}')
```

La última línea causaría un *AttributeError.*

MIEMBROS PÚBLICOS VS NO PÚBLICOS

```python
padre = Padre()
hijo = Hijo()

padre.accede()
hijo.accede()

print(f'Fuera → {padre.__class__.__name__} - {padre.publico}')
print(f'Fuera → {hijo.__class__.__name__} - {hijo.publico}')

print(f'Fuera → {padre.__class__.__name__} - {padre._protegido}')
print(f'Fuera → {hijo.__class__.__name__} - {hijo._protegido}')

# print(f'Fuera → {padre.__class__.__name__} - {padre.__privado}')
# print(f'Fuera → {hijo.__class__.__name__} - {hijo.__privado}')
```

Las dos últimas líneas causarían un *AttributeError.*

Las dos anteriores no causan error, pero deben evitarse por convención. De hecho, PyCharm nos avisa con un *warning*:

EJERCICIO 5.1

OBJETOS CON HERENCIA

- Para saber si un objeto es instancia de una clase determinada, tenemos varias opciones:
  - Usar el atributo *__class__* del objeto (devuelve una referencia a su clase)
  - Usar la función *type(<objeto>)*
  - Usar la función *isinstance(<objeto>, <clase>)*

- La diferencia es que *isinstance()* devuelve *True* si el objeto es de esa clase, o de alguna de sus superclases

- Si queremos comprobar la superclase inmediata de un objeto o clase, usaremos el atributo *__base__* de la clase. Además, podemos encadenar accesos a *__base__* hasta llegar a la clase *object*, de la cual heredan todas las clases en Python

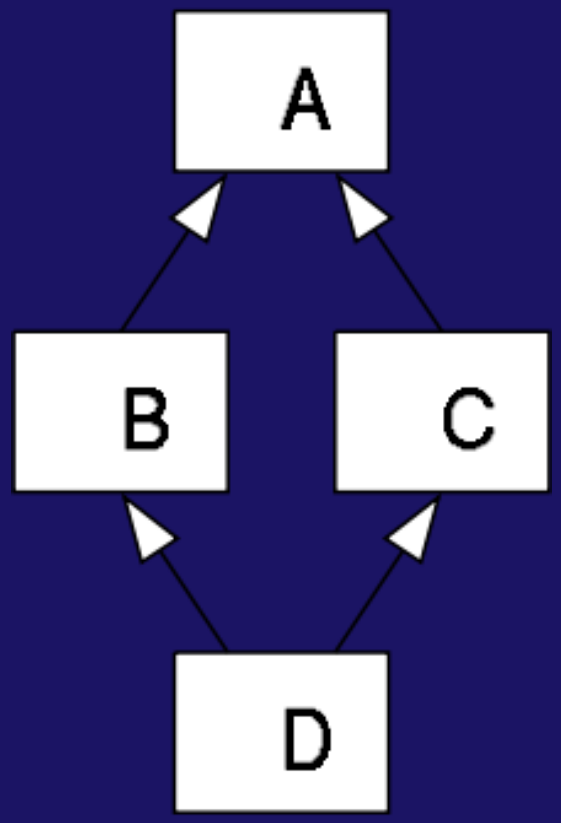
- También disponemos de la función *issubclass(A, B)*, que devuelve *True* si *A* hereda de *B* (aunque no herede de forma directa, ej. A→C→B)

OBJETOS CON HERENCIA

```
>>> hijo.__class__ == Hijo
True

>>> type(hijo) is Hijo
True

>>> isinstance(hijo, Hijo)
True

>>> type(hijo.__class__.__base__) is Padre
True

>>> isinstance(hijo, Padre)
True

>>> issubclass(Hijo, Padre)
True

>>> hijo.__class__
<class '__main__.Hijo'>

>>> hijo.__class__.__base__
<class '__main__.Padre'>

>>> hijo.__class__.__base__.__base__
<class 'object'>

>>> hijo.__class__.__base__.__base__.__base__
None
```

HERENCIA MÚLTIPLE

- En Python, podemos usar herencia múltiple.
- Este tipo de herencia nos permite crear una clase que hereda de varios padres.
- La subclase tendrá acceso a atributos y métodos de todos sus padres.
- La herencia múltiple nos permite reutilizar código de varias clases existentes.
- Sin embargo, debemos gestionar con cuidado la complejidad de la herencia múltiple.
- De lo contrario, pueden aparecer problemas como el problema del diamante.



HERENCIA MÚLTIPLE

```python
# medios_de_transporte.py

class Vehiculo:
    def __init__(self, marca, modelo, color):
        self.marca = marca
        self.modelo = modelo
        self.color = color

    def arrancar(self):
        print("Arrancando el motor...")

    def parar(self):
        print("Parando el motor...")

    def mostrar_especificaciones(self):
        print(f"Marca: {self.marca}")
        print(f"Modelo: {self.modelo}")
        print(f"Color: {self.color}")

class Coche(Vehiculo):
    def conducir(self):
        print("Conduciendo por la carretera...")

class Aeronave(Vehiculo):
    def volar(self):
        print("Volando por el cielo...")

class CocheVolador(Coche, Aeronave):
    pass
```

HERENCIA MÚLTIPLE

```
>>> from medios_de_transporte import CocheVolador

>>> space_flyer = CocheVolador("Space", "Flyer",
"Negro")

>>> space_flyer. mostrar_especificaciones()
Marca: Space
Modelo: Flyer
Color: Negro

>>> space_flyer.arrancar()
Arrancando el motor...

>>> space_flyer.conducir()
Conduciendo por la carretera...

>>> space_flyer.volar()
Volando por el cielo...

>>> space_flyer.parar()
Parando el motor...
```

ORDEN DE RESOLUCIÓN DE METODOS

- Cuando usamos herencia múltiple, podemos encontrar situaciones en las que una clase hereda de dos o más clases que tienen la misma clase base. Esto se conoce como el problema del diamante. El verdadero problema aparece cuando varios padres proporcionan versiones específicas del mismo método. En este caso, sería difícil determinar qué versión de ese método terminará usando la subclase.

- Python soluciona este problema utilizando un orden de resolución de métodos específico (MRO, Method Resolution Order). Entonces, ¿cuál es el orden de resolución de los métodos en Python? El MRO de Python determina qué implementación de un método o atributo usar cuando hay varias versiones de este en una jerarquía de clases.

- En general, Python busca métodos y atributos en el siguiente orden:

    - La clase actual
    - Las superclase más a la izquierda
    - La superclase a continuación, de izquierda a derecha, hasta la última superclase.
    - Las superclases de las clases heredadas.
    - Clase *object*

ORDEN DE RESOLUCIÓN DE METODOS

```
# mro.py

class A:
    def metodo(self):
        print("A.metodo")

class B(A):
    def metodo(self):
        print("B.metodo")

class C(A):
    def metodo(self):
        print("C.metodo")

class D(B, C):
    pass
```

ORDEN DE RESOLUCIÓN DE METODOS

```
>>> from mro import D

>>> D().metodo()
B.metodo

>>> D.__mro__
(
    <class '__main__.D'>,
    <class '__main__.B'>,
    <class '__main__.C'>,
    <class '__main__.A'>,
    <class 'object'>
)
```

A veces, es posible que encontremos relaciones de herencia complejas en las que Python no podrá crear un orden de resolución de métodos consistente. En esos casos, obtendremos un TypeError que señalará el problema.

CONSTRUCTORES CON HERENCIA MÚLTIPLE

```
class A:
    def __init__(self, a):
        self.a = a

class B:
    def __init__(self, b):
        self.b = b

class C:
    def __init__(self, c):
        self.c = c

class Todas(A, B, C):
    def __init__(self, a, b, c):
        A.__init__(self, a)
        B.__init__(self, b)
        C.__init__(self, c)

    def __str__(self):
        return f'{self.a} {self.b} {self.c}'

print(Todas('a', 'b', 'c'))
```

EJERCICIO 5.2

CLASES BASE ABSTRACTAS (ABC) E INTERFACES

- A veces, queremos crear una jerarquía de clases en la que todas las clases implementen una interfaz o API predefinida. En otras palabras, queremos definir el conjunto específico de métodos y atributos públicos que todas las clases de la jerarquía deben implementar (funcionalidad). En Python, podemos hacer esto usando lo que se conoce como clase base abstracta (ABC).

- El módulo *abc* en la biblioteca estándar de Python exporta la clase ABC y otras herramientas relacionadas que podemos usar para definir clases base personalizadas que requieren que todas nuestras subclases implementen interfaces específicas.

- No podemos crear instancias de ABC directamente. Debemos hacer subclases. En cierto sentido, las ABC funcionan como plantillas para que otras clases hereden.

CLASES BASE ABSTRACTAS (ABC) E INTERFACES

```
# formas_abc.py

from abc import ABC, abstractmethod

class Forma(ABC):
    @abstractmethod
    def area(self):
        pass

    @abstractmethod
    def perimetro(self):
        pass
```

CLASES BASE ABSTRACTAS (ABC) E INTERFACES

```
# formas_abc.py

from abc import ABC, abstractmethod
from math import pi

# ...

class Circulo(Forma):
    def __init__(self, radio):
        self.radio = radio

    def area(self):
        return pi * self.radio ** 2
```

CLASES BASE ABSTRACTAS (ABC) E INTERFACES

```
>>> from formas_abc import Circulo

>>> circulo = Circulo(100)
Traceback (most recent call last):
    ...
TypeError: Can't instantiate
abstract class Circulo
    with abstract method perimetro
```

CLASES BASE ABSTRACTAS (ABC) E INTERFACES

```
# formas_abc.py

from abc import ABC, abstractmethod
from math import pi

# ...

class Circulo(Forma):
    def __init__(self, radio):
        self.radio = radio

    def area(self):
        return pi * self.radio ** 2

    def permimetro(self):
        return 2 * pi * self.radio
```

CLASES BASE ABSTRACTAS (ABC) E INTERFACES

```
>>> from formas_abc import Circulo

>>> circlulo = Circulo(100)
>>> circlulo.radio
100
>>> circlulo.area()
31415.926535897932
>>> circlulo.perimetro()
628.3185307179587
```

CLASES BASE ABSTRACTAS (ABC) E INTERFACES

```
# formas_abc.py

from abc import ABC, abstractmethod
from math import pi

# ...

class Cuadrado(Forma):
    def __init__(self, lado):
        self.lado = lado

    def area(self):
        return self.lado ** 2

    def perimetro(self):
        return 4 * self.lado
```

HERENCIA EN CLASES ABSTRACTAS

```
# script.py

from abc import ABC, abstractmethod

class A(ABC):
    @abstractmethod
    def metodo_abstracto_1(self):
        pass

    @abstractmethod
    def metodo_abstracto_2(self):
        pass

    def metodo_concreto(self):
        print('metodo_concreto')

class B(A):
    def metodo_abstracto_1(self):
        print('metodo_abstracto_1')

class C(B):
    def metodo_abstracto_2(self):
        print('metodo_abstracto_2')
```

HERENCIA EN CLASES ABSTRACTAS

```
>>> from script import A, B, C

>>> a = A()
Traceback (most recent call last):
  File "<input>", line 1, in <module>
TypeError: Can't instantiate abstract class A without an
implementation for abstract methods 'metodo_abstracto_1',
'metodo_abstracto_2’

>>> b = B()
Traceback (most recent call last):
  File "<input>", line 1, in <module>
TypeError: Can't instantiate abstract class B without an
implementation for abstract method 'metodo_abstracto_2’

>>> c = C()

>>> c.metodo_concreto()
metodo_concreto

>>>c.metodo_abstracto_1()
metodo_abstracto_1

>>>c.metodo_abstracto_2()
metodo_abstracto_2
```

POLIMORFISMO CON INTERFACES COMUNES

Ya conocemos las clases base abstractas (ABCs) y cómo aplicarlas para usar una interfaz pública común en varias clases relacionadas. Tener un conjunto de clases para implementar la misma interfaz con comportamientos específicos para clases concretas es una excelente manera de desbloquear el polimorfismo.

El polimorfismo se da cuando podemos usar objetos de diferentes clases indistintamente porque comparten una interfaz común. Por ejemplo, las cadenas, listas y tuplas de Python son todos tipos de datos de secuencia. Esto significa que implementan una interfaz común a todas las secuencias.

Debido a esta interfaz común, podemos usarlos de manera similar. Por ejemplo, podemos:

- Usarlos en bucles porque proporcionan el método *.__iter__()*
- Acceder a sus elementos por índice porque implementan el método *.__getitem__()*
- Conocer su número de elementos porque incluyen el método *.__len__()*

Podemos ejecutar todas estas operaciones y más sin preocuparnos de qué tipo específico estamos utilizando realmente en nuestro código. Eso es posible gracias al polimorfismo.

POLIMORFISMO CON INTERFACES COMUNES

```
>>> mensaje = "Hola!"
>>> numeros = [1, 2, 3]
>>> letras = ("A", "B", "C")

>>> len(mensaje)
6
>>> len(numeros)
3
>>> len(letras)
3
```

POLIMORFISMO CON INTERFACES COMUNES

```
from formas_abc import Circulo, Cuadrado

formas = [Circulo(10), Circulo(20), Cuadrado(2),\
          Cuadrado(4)]

for forma in formas:
    print(f'{type(forma).__name__}, '
          f'Area = {forma.area()}, '
          f'Perimetro = {forma.perimetro()}')


# Salida

Circulo, Area = 314.1592653589793, Perimetro = 62.83185307179586
Circulo, Area = 1256.6370614359173, Perimetro = 125.66370614359172
Cuadrado, Area = 4, Perimetro = 8
Cuadrado, Area = 16, Perimetro = 16
```

EJERCICIO 5.3